

Applications: Components, Cycles, and Topological Order

Programming and Data Structures — Week 11, Lecture 3

Dr. Innocent Nyalala

Objectives

By the end of this lecture, you will be able to find all connected components of a graph using nothing more than repeated application of Lecture 2's traversal, and explain why this requires no genuinely new algorithm at all; detect a cycle in an undirected graph and in a directed graph, using two structurally different DFS-based techniques, each specific to that graph type; implement topological sort using DFS, and explain precisely why this operation only makes sense for directed acyclic graphs; and trace all three applications completely on concrete example graphs.

Outline

Today covers an island-hopping analogy for connected components; finding connected components via nothing more than repeated traversal; cycle detection in an undirected graph, traced completely; cycle detection in a directed graph, and why the undirected method fails there; topological sort via a modified DFS, traced completely; a precise explanation of why topological sort is only meaningful for directed acyclic graphs; the MTech-track alternative, Kahn's algorithm, computing topological order via BFS instead of DFS; an in-class quiz; and a summary.

The island-hopping analogy

A graph can be disconnected, consisting of several separate “islands” of mutually reachable nodes with no path connecting different islands. A single BFS or DFS run, starting from one node, only ever explores that node's own island — nodes belonging to other, unconnected islands are never discovered or visited at all, regardless of how thoroughly the first island is explored. Finding every island in the graph requires starting a fresh traversal from any not-yet-visited node whenever one is found, repeating this process until every node in the entire graph has been visited by some traversal.

Connected components: repeated traversal, nothing new

```
def connected_components(graph, n):
    visited = set()
    components = []
    for node in range(n):
        if node not in visited:
            component = bfs(graph, node)      # Lecture 2's BFS, unchanged
            visited.update(component)
            components.append(component)
    return components
```

This algorithm invents nothing new whatsoever — it is a simple loop that calls Lecture 2's `bfs` (or equivalently `dfs`) repeatedly, once for every node not yet visited by a previous call. Each individual call discovers exactly one complete connected component, since traversal by definition reaches every node reachable from its starting point and nothing beyond it. Components never overlap, because every node discovered by one traversal is immediately marked `visited`, preventing any later traversal from rediscovering it. This is a clean demonstration that a correct “algorithm” is sometimes nothing more than correctly reusing an already-built tool, applied systematically.

Tracing connected components completely

Graph: `adj[0]=[1]`, `adj[1]=[0]`, `adj[2]=[3]`, `adj[3]=[2,4]`, `adj[4]=[3]`, `adj[5]=[]`

node 0: not visited → `bfs(0) = [0,1]` → `components=[[0,1]]`, `visited={0,1}`
node 1: visited, skip
node 2: not visited → `bfs(2) = [2,3,4]` → `components=[[0,1],[2,3,4]]`, `visited={0,1,2,3,4}`
node 3: visited, skip
node 4: visited, skip
node 5: not visited → `bfs(5) = [5]` → `components=[[0,1],[2,3,4],[5]]`, `visited={0,1,2,3,4,5}`

Result: 3 connected components - `{0,1}`, `{2,3,4}`, `{5}`

Tracing this 6-node graph with three genuinely separate islands: nodes 0 and 1 form one component, connected only to each other; nodes 2, 3, and 4 form a second component; and node 5, with no edges at all, forms a third, singleton component entirely on its own. The loop correctly discovers all three, in order, skipping every node already claimed by an earlier component.

Cycle detection in an undirected graph

```
def has_cycle_undirected(graph, n):
    visited = set()
    def dfs_check(node, parent):
        visited.add(node)
        for neighbor, weight in graph.adj[node]:
            if neighbor not in visited:
                if dfs_check(neighbor, node):
                    return True
            elif neighbor != parent:          # visited AND not where we just came from
                return True                  # found a genuine cycle
        return False
    for node in range(n):
        if node not in visited:
            if dfs_check(node, -1):
                return True
    return False
```

The key insight for undirected cycle detection is explicitly tracking the parent node during DFS. Encountering an already-visited neighbor is only genuine evidence of a cycle if that neighbor is not the immediate parent the current node was reached from — without this specific check, every single edge would falsely register as a cycle, since an undirected edge between u and v means v appears in u 's neighbor list and u appears in v 's, so DFS would immediately “see” the parent again through the edge just traversed, which is not a real cycle at all.

Tracing undirected cycle detection

Graph: $\text{adj}[0]=[1,2]$, $\text{adj}[1]=[0,2]$, $\text{adj}[2]=[0,1]$ -- a genuine triangle (cycle)

```
dfs_check(0, parent=-1): visited={0}
  neighbor 1: not visited → dfs_check(1, parent=0): visited={0,1}
    neighbor 0: visited AND is parent → skip, not a cycle
    neighbor 2: not visited → dfs_check(2, parent=1): visited={0,1,2}
      neighbor 0: visited AND NOT parent (parent is 1) → CYCLE DETECTED, return True
```

Tracing cycle detection on a genuine 3-node triangle: starting DFS at node 0, then node 1 (parent 0), then node 2 (parent 1), the algorithm reaches node 2's neighbor 0 — already visited, but critically, 0 is not node 2's parent (node 2's parent is 1). This correctly identifies a genuine cycle, distinct from node 1's earlier encounter with node 0, which was correctly skipped since 0 was node 1's actual parent.

Cycle detection in a directed graph: why undirected's method fails

A directed graph's edges only permit travel in one direction, so the parent-tracking trick used for undirected graphs does not apply — a directed cycle can revisit an earlier node entirely legitimately, without ever traversing the same edge backward, since a directed edge has no “backward” direction at all. The correct fix tracks which nodes are currently on the active recursion stack, representing the current DFS path from the root, rather than merely which nodes have ever been visited overall. A genuine cycle exists exactly when some edge points to a node still on that active path — this distinction between “visited at some point” and “currently on the path” is the entire structural difference separating directed cycle detection from undirected.

Directed cycle detection, in code

```
def has_cycle_directed(graph, n):
    WHITE, GRAY, BLACK = 0, 1, 2          # unvisited, on current path, fully done
    color = [WHITE] * n
    def dfs_check(node):
        color[node] = GRAY
        for neighbor, weight in graph.adj[node]:
            if color[neighbor] == GRAY:
                return True                # edge to a node on the CURRENT path - genuine cycle
            if color[neighbor] == WHITE and dfs_check(neighbor):
                return True
        color[node] = BLACK
        return False
    return any(dfs_check(node) for node in range(n) if color[node] == WHITE)
```

This three-color scheme — white for unvisited, gray for currently on the active DFS path, black for fully processed and no longer on the path — directly implements the distinction from the previous slide. An edge to a gray node is immediate proof of a cycle, since it points back to an ancestor still being processed on the current path. An edge to a black node is safe, since that node and its entire subtree have already been fully explored and are no longer part of any currently active path.

Topological sort: DFS with a twist

```
def topological_sort(graph, n):
    visited = set()
    order = []
```

```

def dfs_visit(node):
    visited.add(node)
    for neighbor, weight in graph.adj[node]:
        if neighbor not in visited:
            dfs_visit(neighbor)
    order.append(node)          # append AFTER visiting all neighbors - the twist
for node in range(n):
    if node not in visited:
        dfs_visit(node)
return order[::-1]            # reverse at the very end

```

A node is appended to the output list only after every node it points to has been fully processed — this is precisely Week 7 Lecture 1's postorder traversal idea (process children before the parent), applied here to a graph rather than a tree. Reversing the resulting list at the very end places every node before anything that depends on it, which is exactly the defining property of a topological order: for every directed edge from u to v , u appears before v in the final output.

Tracing topological sort completely

Graph (course prerequisites, directed): `adj[0]=[1,2]`, `adj[1]=[3]`, `adj[2]=[3]`, `adj[3]=[]`
(0="Intro Programming", 1="Data Structures", 2="Discrete Math", 3="Algorithms")

```

dfs_visit(0): visited={0}
dfs_visit(1): visited={0,1}
dfs_visit(3): visited={0,1,3}. No neighbors. order=[3]
order=[3,1]
dfs_visit(2): visited={0,1,3,2}
(3 already visited, skip)
order=[3,1,2]
order=[3,1,2,0]

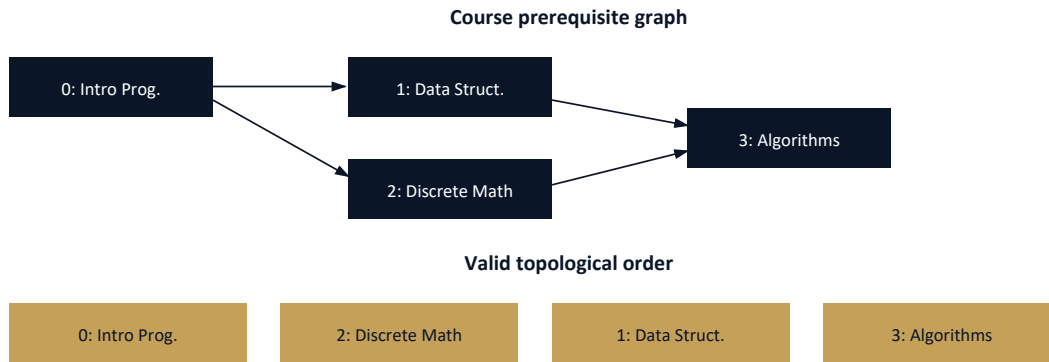
```

Reversed: [0, 2, 1, 3]

Valid topological order: Intro Programming, Discrete Math, Data Structures, Algorithms

Tracing topological sort on a small course-prerequisite graph: DFS from node 0 reaches node 1, then node 3 (a sink with no outgoing edges, appended first), then backtracks to append 1, then explores node 2 (finding 3 already visited), appending 2, and finally appending 0. Reversing the collected order [3, 1, 2, 0] gives [0, 2, 1, 3] — a valid course-taking order where every prerequisite appears before the course that depends on it.

The picture: the DAG and its topological order



Every arrow above points left-to-right in this ordering — the defining property of a valid topological sort.

Every arrow in the original graph points from an earlier position to a later position in the linear ordering shown below it — this is the entire, complete definition of what makes an ordering “topological,” made visually checkable on this specific example.

Why topological sort requires a DAG

If a directed graph contains a cycle, no valid linear ordering can possibly exist: some node within the cycle would necessarily need to appear both before and after another node in that same cycle, an outright contradiction. “DAG,” directed acyclic graph, names precisely the class of graphs for which topological sort is even a meaningful, well-posed question in the first place. In practice, this means always running this lecture’s directed cycle-detection algorithm first, and only attempting topological sort once the graph is confirmed acyclic — running topological sort on a graph with an undetected cycle produces a silently incorrect, incomplete, or misleading result rather than a clean error.

MTech addition — Kahn’s algorithm: topological sort via BFS

```
def kahn_topological_sort(graph, n):
    in_degree = [0] * n
    for u in range(n):
        for v, weight in graph.adj[u]:
            in_degree[v] += 1
    queue = deque([node for node in range(n) if in_degree[node] == 0])
    order = []
    while queue:
        node = queue.popleft()
```

```

order.append(node)
for neighbor, weight in graph.adj[node]:
    in_degree[neighbor] -= 1
    if in_degree[neighbor] == 0:
        queue.append(neighbor)
return order if len(order) == n else None    # None signals a cycle was present

```

Kahn’s algorithm takes a structurally different approach: it repeatedly removes nodes that currently have zero remaining incoming edges, tracking this with Week 3’s queue in exactly BFS’s structure, and decrements the in-degree of each removed node’s neighbors as it goes. If the queue empties while nodes still remain unprocessed, a cycle must exist among those remaining nodes, since none of them could ever reach zero in-degree — this doubles as a free cycle-detection check, without needing this lecture’s separate three-color DFS method. Kahn’s algorithm costs the same $O(V + E)$ as the DFS-based version, a genuinely different mechanism arriving at the identical asymptotic cost.

Exercise

As an exercise, implement `connected_components`, `has_cycle_undirected`, `has_cycle_directed`, and `topological_sort` exactly as presented in this lecture, verifying each against this lecture’s traced examples. Separately, implement `kahn_topological_sort` and confirm it produces a valid topological order on the course-prerequisite example — note that it need not match the DFS-based version’s exact output order, only that every directed edge’s before/after relationship is correctly preserved in whichever valid order it produces.

Real-world use of these three applications

Connected components identify separate clusters in a social network graph, and detect isolated, unreachable machines in a network topology map. Cycle detection underlies deadlock detection in operating systems, where a resource-allocation graph’s cycle indicates processes stuck waiting on each other indefinitely, and detects circular dependencies in software build systems, which would otherwise attempt to build a component before its own prerequisite is ready. Topological sort underlies real build systems — Makefiles and package managers ordering compilation and installation steps correctly — and course-prerequisite scheduling, exactly this lecture’s own worked example, applied at real institutional scale.

Quiz — component count

Given a graph with `adj[0]=[1]`, `adj[1]=[0]`, `adj[2]=[]`, `adj[3]=[4]`, and `adj[4]=[3]`, determine how many connected components it has, before checking the answer.

Quiz — answer

Nodes 0 and 1 are mutually connected, forming one component. Node 2 has no edges at all, forming its own singleton component. Nodes 3 and 4 are mutually connected, forming a third component. The total is 3 connected components — directly analogous to this lecture’s earlier traced example, confirming the same counting logic applies regardless of the specific graph’s shape.

A second worked cycle-detection trace, on an acyclic directed graph

Graph: `adj[0]=[1,2]`, `adj[1]=[3]`, `adj[2]=[3]`, `adj[3]=[]` (this lecture's course DAG)

```
dfs_check(0): color[0]=GRAY
  dfs_check(1): color[1]=GRAY
    dfs_check(3): color[3]=GRAY. No neighbors. color[3]=BLACK. return False
  color[1]=BLACK. return False
  dfs_check(2): color[2]=GRAY
    neighbor 3: color[3]=BLACK (already done, safe) → no cycle
  color[2]=BLACK. return False
color[0]=BLACK. return False
```

Result: no cycle - confirming this graph is a genuine DAG, safe for topological sort

Running directed cycle detection on this lecture’s own course- prerequisite graph confirms it is genuinely acyclic: every neighbor encountered is either white (unvisited, explored recursively) or black (already fully processed, safe), and no gray (currently-on-path) node is ever encountered as a neighbor. This confirms, rather than merely assumes, that the earlier topological sort trace was valid to perform at all — exactly the “check first” discipline this lecture’s earlier slide insisted on.

Summary

Connected components are found through nothing more than repeated application of Lecture 2’s traversal, starting fresh from every not-yet- visited node. Undirected cycle detection tracks the immediate parent to distinguish a genuine cycle from simply re-traversing the same edge backward; directed cycle detection instead tracks which nodes remain on the active recursion-stack path, using a white/gray/black coloring scheme, since the parent trick does not apply to one-directional edges. Topological sort is DFS’s postorder traversal, reversed, meaningful only for directed acyclic graphs, and only valid to run after confirming the graph is genuinely acyclic. Kahn’s algorithm computes the identical result via BFS and in-degree counting instead, doubling as a cycle check for free. Next lecture closes this course’s algorithmic-techniques arc with parallel and randomized approaches.